Deliverable 1: Term Project

CSI 2132 A00 - Databases I

Winter 2026

School of Electrical Engineering and Computer Science,
University of Ottawa

Course Coordinator: Dr. Verena Kantere

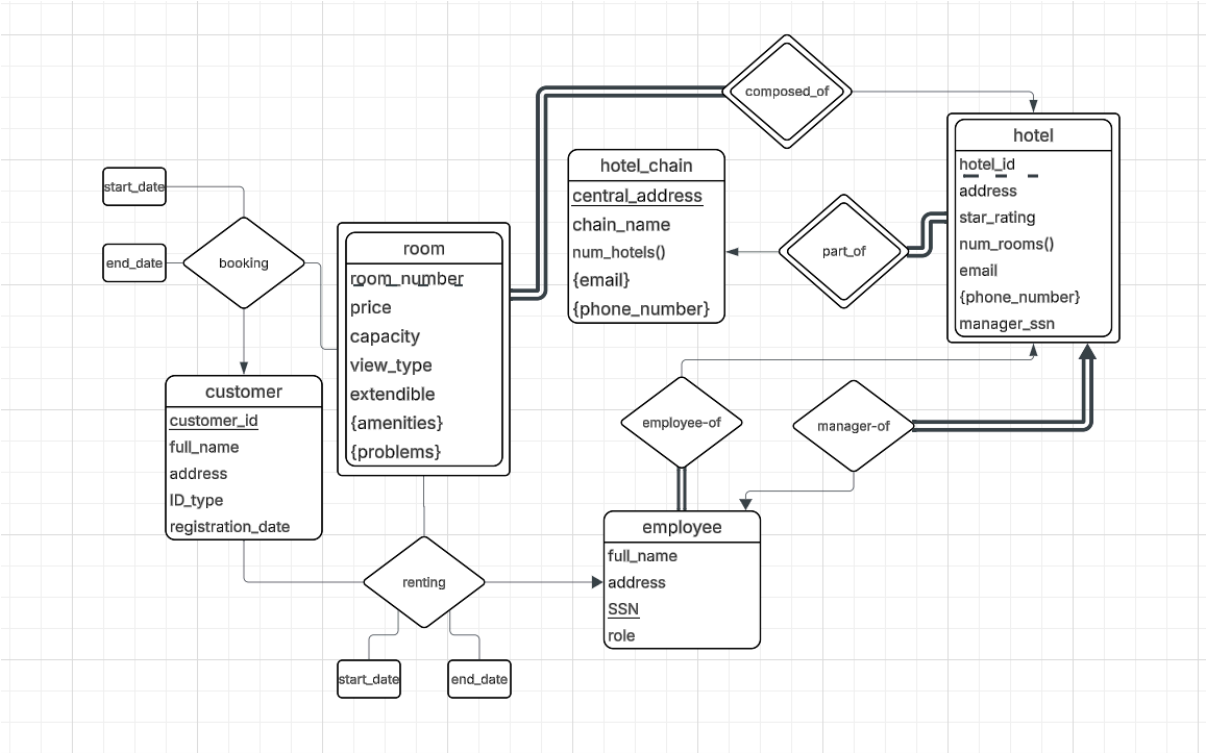
Group 90

Edward-Joseph Fattouch (300353595)

Ipinoluwa Shobayo (300381875)

Victor Situ (300349206)

Submission Date: February 25, 2026



Explanation

The ER diagram is structured around a strong hierarchical dependency between **hotel_chain**, **hotel**, and **room**. The identifying relationships (“composed_of” and “part_of”) and the double lines indicate total participation and existence dependency. A hotel cannot exist without belonging to a hotel chain, and a room cannot exist without belonging to a specific hotel. This enforces structural integrity directly at the conceptual level and reflects the real-world organization of hotel networks.

Multi-valued attributes are clearly separated in the diagram using curly braces. For example, hotel chains can have multiple emails and phone numbers, hotels can have multiple phone numbers, and rooms can have multiple amenities and problems. Modelling these as multi-valued attributes (instead of single repeated fields) keeps the design normalized and avoids redundancy. Derived attributes such as **`num_hotels()`** and **`num_rooms()`** are also shown explicitly, indicating that they are calculated rather than stored.

Operationally, the model distinguishes between **booking** and **renting** as two separate relationships. Booking connects the customer and the room with start and end dates, representing a reservation before arrival. Renting also connects the customer and the room with start and end dates, but additionally links to the employee, capturing who handled the check-in. This separation allows the system to support both reservations and walk-in customers while maintaining accountability.

The employment structure is also clearly modelled. The “employee-of” relationship assigns employees to a specific hotel, while the “manager-of” relationship ensures that each hotel has exactly one manager. Since the manager is modelled as a relationship rather than just an attribute, the diagram enforces that the manager must be an employee of that hotel.

Overall, the ER design is consistent with the stated assumptions: strict existence dependency, clear operational processes, normalized multi-valued attributes, and explicit enforcement of managerial and employment constraints.

Assumptions:

- We only care about hotels in North America
- Assume phone numbers have no extensions
- Customers can have multiple bookings.

Relational Schemas:

Hotel_Chain(central_address, chain_name)
Hotel_Chain_Phone(central_address, phone_number)
Hotel_Chain_Email(central_address, email_address)
Hotel(hotel_id, hotel_chain_central_address, address, star_rating, email, manager_ssn)
Hotel_Phone(hotel_id, phone_number)
Room(room_number, hotel_id, price, capacity, view_type, extendible)
Room_Amenities(room_number, hotel_id, amenity_type)
Room_Problems(room_number, hotel_id, problem_type)
Employee(ssn, hotel_id, full_name, address, role)
Customer(customer_id, full_name, address, id_type, registration_date)
Booking(customer_id, hotel_id, room_number, start_date, end_date)
Renting(customer_id, hotel_id, room_number, employee_ssn, start_date, end_date)

Foreign Keys:

Hotel_Chain_Phone.central_address to Hotel_Chain.central_address
Hotel_Chain_Email.central_address to Hotel_Chain.central_address
Hotel.hotel_chain_central_address to Hotel_Chain.central_address
Hotel.manager_ssn to Employee.ssn
Hotel_Phone.hotel_id to Hotel.hotel_id
Room.hotel_id to Hotel.hotel_id
Room_Amenities.room_number to Room.room_number
Room_Amenities.hotel_id to Room.hotel_id
Room_Problems.room_number to Room.room_number
Room_Problems.hotel_id to Room.hotel_id
Employee.hotel_id to Hotel.hotel_id
Booking.customer_id to Customer.customer_id
Booking.hotel_id to Hotel.hotel_id
Booking.room_number to Room.room_number
Renting.customer_id to Customer.customer_id
Renting.hotel_id to Hotel.hotel_id
Renting.room_number to Room.room_number
Renting.employee_ssn to Employee.ssn

Explanation

This relational database schema is designed to enforce the strict hierarchy of a hotel network. To maintain data integrity, the architecture relies on foreign key constraints and composite primary keys to define weak entities. Specifically, the schema dictates that a Room cannot exist independently of a Hotel (linked via hotel_id), and a Hotel must be permanently anchored to a Hotel_Chain (linked via hotel_chain_central_address). Beyond enforcing these structural dependencies, the design enforces atomicity by isolating multi-valued properties. By moving variable data like Hotel_Chain_Phone, Room_Amenities, and Room_Problems into their own dedicated tables, the database avoids redundancy and updates anomalies.

Another major strength of this design is how it handles guest transactions. By explicitly separating Booking (the intent to stay) from Renting (the finalized stay), the database mirrors real-world hotel operations. The Renting table is specifically structured to capture the employee_ssn, ensuring accountability for whoever processes the check-in. This separation provides the flexibility needed to handle both customers arriving with a pre-planned reservation and those walking in directly from the street.

Finally, the schema is tailored to the project's specific operational assumptions. The rule that every hotel is overseen by exactly one manager is directly modeled by placing the manager_ssn foreign key inside the core Hotel table. Likewise, attributes explicitly defined as singular, such as an employee having only one role or a hotel having a single email, are embedded directly into their main tables. This keeps the overall structure efficient, logically sound, and optimized for everyday queries.

Attribute Domains and Attribute Constraints:

Foreign Keys previously stated above.

Relation Schema	Attributes with Constraints	Explanation
Hotel_Chain	Central_address is a unique string and cannot be null (PK)	Each hotel chain has a unique address and can be distinguished from it. Hotel chains have to be named.
Hotel_Chain	Chain_name is a string and cannot be null	
Hotel_Chain_Phone	Central_address is a string and cannot be null (PK)	Composite PK to prevent duplicate phone numbers for hotel chains.
Hotel_Chain_Phone	Phone_number is a string of integers and has a length of 10 and cannot be null (PK)	
Hotel_Chain_Email	Central_address is a string and cannot be null (PK)	Composite PK to prevent duplicate emails for hotels.
Hotel_Chain_Email	Email_address is a string that must include @ and cannot be null (PK)	

Hotel	Hotel_id is an integer and cannot be null (PK)	Composite PK since hotels in different hotel chains could have the same ID. Hotels must have an address thus cannot be empty. Each hotel is linked to the SSN of its manager and can be temporarily null prior to assigning the manager.
Hotel	Hotel_chain_central_address is a string and cannot be null (PK)	
Hotel	Address is a string and cannot be null	
Hotel	Star_rating is an integer between 1 and 5	
Hotel	Email is a string with an @ symbol and cannot be null	
Hotel	Manager_ssn is a unique integer	
Hotel_Phone	Hotel_id is an integer and cannot be null (PK)	Composite PK to prevent duplicate phone numbers for hotels.
Hotel_Phone	Phone_number is a string of integers and has a length of 10 and cannot be null (PK)	
Room	Room_number is an integer and cannot be null (PK)	Composite PK since different hotels can have rooms with the same room number. Rooms can be free in special circumstances but never less than 0. View type is limited to sea or mountain or a null value meaning there is no special view.
Room	Hotel_id is an integer and cannot be null (PK)	
Room	Price is an integer greater than or equal to 0	
Room	capacity is a string and must be one of ('single', 'double', 'suite', 'family') and cannot be null	
Room	view_type is a string and must be one of ('sea', 'mountain')	
Room	Extendible is a boolean True or False and cannot be null	
Room_Amenities	Room_number is an integer and cannot be null (PK)	Composite PK to differentiate the same named amenities in different rooms.
Room_Amenities	Hotel_id is an integer and cannot be null (PK)	
Room_Amenities	amenity_type is a string (Ex: 'TV', 'WiFi', 'AirCondition') and cannot be null (PK)	
Room_Problems	Room_number is an integer and cannot be null (PK)	Composite PK to differentiate the same named problems in different rooms.
Room_Problems	Hotel_id is an integer and	

	cannot be null (PK)	
Room_Problems	problem_type is a string (Ex: ‘Damaged shelf’, ‘Creaky Door’) and cannot be null (PK)	
Employee	ssn is a unique integer and cannot be null (PK)	Every employee has a unique SSN. Employees have a role representing their job at the hotel.
Employee	hotel_id is an integer and cannot be null	
Employee	full_name is a string and cannot be null	
Employee	address is a string and cannot be null	
Employee	role is a string and must be one of ('Manager', ‘Cook’, etc.)	
Customer	customer_id is a unique integer and cannot be null (PK)	Customer ID is unique to each customer. Every customer has to show a type of ID so it cannot be null. The registration date can’t be in the future.
Customer	full_name is a string and cannot be null	
Customer	address is a string and cannot be null	
Customer	id_type is a string and must be one of ('SSN', 'SIN', 'DriverLicense', 'Passport') and cannot be null	
Customer	registration_date is a date and must be less than or equal to the current date	
Booking	customer_id is a unique integer and cannot be null (PK)	Composite PK to make each booking unique to the customer booking the specific room of the hotel, also differentiates from bookings of rooms at the same hotel by the same customer. End_date can be null if the customer does not have a determined end date and is paying by day.
Booking	hotel_id is an integer and cannot be null (PK)	
Booking	Room_number is an integer and cannot be null (PK)	
Booking	Start_date is a string and cannot be null	
Booking	End_date is a string and must be after start_date	
Renting	customer_id is a unique integer and cannot be null (PK)	Composite PK to make each renting unique to the customer renting the specific room of the hotel and the employee that managed the renting, also differentiates from bookings of
Renting	hotel_id is an integer and cannot be null (PK)	

Renting	Room_number is an integer and cannot be null (PK)	rooms at the same hotel by the same customer. End_date can be null if the customer does not have a determined end date and is paying by day.
Renting	Employee_ssn is an integer and cannot be null (PK)	
Renting	Start_date is a string and cannot be null	
Renting	End_date is a string and must be after start_date	

User-Defined Constraints:

1. **Every hotel must have exactly one employee whose role is 'Manager'.**
   Ensures each hotel has a single designated manager responsible for its operations.
2. **An employee can only work at one hotel.**
   Prevents an employee from being assigned to multiple hotels simultaneously.
3. **A room cannot have overlapping bookings for the same dates.**
   Prevents double reservation of the same room during overlapping time periods.
4. **A room cannot have overlapping rentings for the same dates.**
   Ensures a room cannot be occupied by more than one customer at the same time.
5. **A renting can only be created by an employee who works at that same hotel.**
   Guarantees that check-ins are processed only by authorized staff of that hotel.
6. **When a hotel chain is deleted, all hotels belonging to it must also be deleted.**
   Maintains hierarchical integrity by cascading deletions downward.
7. **When a hotel is deleted, all its rooms must also be deleted.**
   Ensures no room exists without its parent hotel.
8. **Archived bookings and rentings must remain in the database even if the related customer or room is deleted.**
   Preserves historical transaction data for auditing and record-keeping.
9. **Hotel chains must have unique addresses.**
   Prevents duplicate identification of hotel chains.
10. **Every hotel has only one email address.**
    Enforces the assumption that each hotel uses a single official contact email.
11. **Employees can only have one role.**
    Ensures each employee is assigned a single, well-defined position within the hotel.
12.